

Assignment 2: AI Portfolio Advisor

1. Main Questions Answered in This Prototype

Assignment 1 delivered a portfolio risk analytics platform with ML-based macro predictions, sentiment analysis, and LLM-generated advice (single-call summarization via Claude Haiku). The feedback highlighted strong execution but noted that the LLM usage was straightforward and documentation lacked depth on model evaluation. Assignment 2 addresses these gaps by asking and answering three questions:

- Q1: Can an LLM act as an autonomous financial agent?** I built an **agentic chatbot** (Claude Sonnet) that autonomously decides which of 6 tools to call (portfolio summary, asset prices, risk metrics, what-if simulation, market news, macro predictions), executes them in a multi-turn loop (up to 8 iterations), feeds results back to itself, and synthesizes a final answer. The LLM controls the execution flow, not the developer.
- Q2: Can an LLM drive a multi-step decision chain end-to-end?** The **AI Autopilot** (Suggestions view) implements an orchestrated 3-step chain: (1) Claude proposes 3 trades as structured JSON, validated by schema checks; (2) Python simulates each trade using the what-if engine, computing new volatility, Sharpe, and drawdown; (3) Claude synthesizes all simulations into a ranked recommendation. Each call is enriched with a **cross-component signal pipeline** macro predictions, NLP sentiment, efficient frontier weights, stress test results, and backtest metrics all injected via `st.session_state`.
- Q3: Can an LLM generate structured scenario parameters for quantitative simulation?** The **Stress Test** view sends a user-described scenario to Claude, which returns validated JSON with per-holding drawdown percentages, sector rationales, correlation spike estimates, and overall market impact. Python then applies these quantitatively the LLM generates assumptions, not final numbers.

Together, these three patterns represent the full spectrum of LLM integration complexity: single-call (A1), structured output, agentic tool-use, and orchestrated multi-step chains.

2. LLM Integration Patterns (4 Distinct Patterns)

Pattern	Model	View	Non-Trivial Aspect
Single-call summarization	Haiku	Dashboard, News, Macro	Prompt enriched with full risk metrics + multi-source signals
Agentic tool-use loop	Sonnet	AI Chat	6 tools, multi-turn (up to 8 iterations), Claude controls flow autonomously
Structured JSON generation	Sonnet	Stress Test	Schema-validated JSON output; Python applies params quantitatively
Orchestrated multi-step chain	Sonnet	Suggestions (Autopilot)	3-step: LLM proposes > Python simulates > LLM synthesizes recommendation

3. Additional Improvements (Non-LLM)

Beyond the LLM features, the prototype received major enhancements in response to Assignment 1 feedback:

- ML Transparency tab:** A dedicated view displaying full model evaluation classification reports, ROC curves, PR curves, calibration diagrams, 5-fold CV results, learning curves, feature importance (Gini + permutation), feature correlation heatmaps, and live model introspection. All computed from saved evaluation artifacts.
- Efficient Frontier:** Monte Carlo simulation (5,000 random portfolios) + MPT optimization (max Sharpe, min volatility) using `scipy.optimize.minimize`. Displays current portfolio position vs. the efficient frontier.
- Backtest engine:** Historical portfolio performance vs. SPY benchmark with annualized return, Sharpe, max drawdown, and Calmar ratio. Supports 1M to 5Y lookback.
- CVaR & dynamic risk-free rate:** Conditional Value at Risk (expected loss in worst 5% of days) added to all risk calculations. Sharpe ratio uses live US 10Y Treasury yield via `yfinance`.
- LLM usage tracking:** Persistent cost/token/latency accumulator across all calls (Haiku \$0.80/\$4.00, Sonnet \$3.00/\$15.00 per M tokens), displayed in sidebar.
- ELI10 mode:** Toggle that switches all LLM system prompts to child-friendly language with analogies, applied consistently across all 4 LLM patterns.

4. Main Difficulties Encountered

- **Agentic tool-use loop reliability:** Claude occasionally called tools with malformed inputs or entered infinite loops. Mitigated with an 8-iteration hard cap, strict JSON schema validation on each tool call, and a fallback that extracts the last text response if the loop exhausts.
- **Structured JSON generation:** Even with explicit instructions, Claude sometimes wraps output in code fences or adds commentary. Implemented robust post-processing: strip markdown fences, attempt JSON parse, and return structured error messages on failure rather than crashing.
- **Cross-component signal pipeline timing:** Signals (macro, sentiment, frontier, stress, backtest) are computed in separate views. If the user has not visited a view, that signal is None. Used defensive try/except and conditional prompt injection signals only added when available.
- **Train-serve skew prevention:** Feature engineering was initially duplicated. Refactored into centralized `ml_pipeline/features.py` with canonical feature list, shared `build_macro_features()` function, and version-aware selection (v1=12 features, v2=16 features).
- **yfinance API instability:** The DXY ticker changes between DX-Y.NYB and DX=F. Implemented fallback chain + MultiIndex column handling for yfinance ≥ 1.2 .

5. How I Leveraged AI

AI was used as a development partner throughout the project, not just as a code generator:

- **Architecture design:** Used Claude as a thought partner to design the cross-component signal pipeline mapping which views produce/consume which signals. The `session_state` registry pattern emerged from structured brainstorming.
- **Prompt engineering iteration:** The Autopilot's 3-step chain required significant refinement. Initial Step 1 prompt produced invalid JSON ~40% of the time. Through iterative testing, added schema examples, 'Rules' sections, and 'no markdown fences' reducing failure to under 5%.
- **Test-driven development:** Generated comprehensive test suites (tests/ directory) covering portfolio metrics, stress test engine, chatbot tools, and AI advisor. Tests mock the Anthropic API to verify tool dispatch and error handling.
- **ML evaluation pipeline:** Designed the pattern of saving `y_test`, `y_pred`, `y_prob` plus CV scores, learning curves, and permutation importances as a single joblib artifact enabling the ML Transparency tab without re-training.
- **Documentation:** This report was drafted with AI assistance, ensuring all claims are grounded in the actual codebase.

6. System Architecture

Layer	Technologies
Frontend	Streamlit (10 interactive views, custom CSS, ELI10 toggle)
Visualizations	Plotly Graph Objects / Express (candlestick, scatter, heatmap, gauge)
Financial Data	yfinance (live prices, historical data, news, asset metadata), 5-min cache
ML / NLP	scikit-learn, XGBoost, joblib, TF-IDF, GridSearchCV, TimeSeriesSplit
LLM API	Anthropic Claude SDK (Haiku for summaries, Sonnet for agentic + structured)
Risk Engine	Covariance-based volatility, Beta, HHI, Sharpe, CVaR, Max Drawdown
Signal Pipeline	<code>st.session_state</code> registry (5 cross-component signals)

7. Addressing Assignment 1 Feedback

The Assignment 1 feedback specifically noted: "provides limited detail on model evaluation, feature engineering, and empirical performance." This was addressed comprehensively:

Feedback Point	How Addressed in Assignment 2
Model evaluation depth	Full ML Transparency tab: ROC, PR, calibration curves, confusion matrices, 5-fold CV scores, learning curves all from live evaluation artifacts
Feature engineering clarity	12-feature table with source, description, and financial rationale; centralized in <code>features.py</code> with versioning (v1/v2)
Empirical performance	All-model AUC comparison table; permutation importance vs Gini importance; sentiment accuracy + top TF-IDF words
Straightforward LLM usage	4 distinct LLM patterns (see Section 2); agentic tool-use and orchestrated chain are explicitly non-straightforward

Repository: github.com/paugratacosfuste/portfolio_manager | Live Demo: portfolio-manager-pau.streamlit.app | Built with: Streamlit, Plotly, scikit-learn, XGBoost, Anthropic Claude API